Beat-by-Beat Heart Rate and Heart Rate Variability Analysis

Introduction

An ECG signal recorded using the ECG instrumentation amplifier built during the laboratory exercise was analyzed to compute beat-by-beat heart rate (HR) and heart rate variability (HRV). The ECG was recorded in a Lead-I configuration with right-leg grounding for approximately one minute and digitized at a sampling rate of:

$$f_s = 1000 \text{ Hz}$$

All analysis was implemented in Python.

1. Beat-by-Beat Heart Rate Measurement

R-Peak Detection

R-peaks were detected from the ECG using a threshold-based algorithm:

1. A voltage threshold of 750 mV was used to identify candidate QRS complexes.
2. The local maximum within each threshold crossing was selected as the R-peak.
3. A refractory period of 0.25 s prevented multiple detections within the same QRS complex.
4. Missed beats were recovered by identifying abnormally large RR intervals and searching for the largest peak within that interval (as some of the R-peaks were below the set 750 mV).

A total of 72 R-peaks were detected in the one-minute ECG recording.

ECG with Detected Peaks

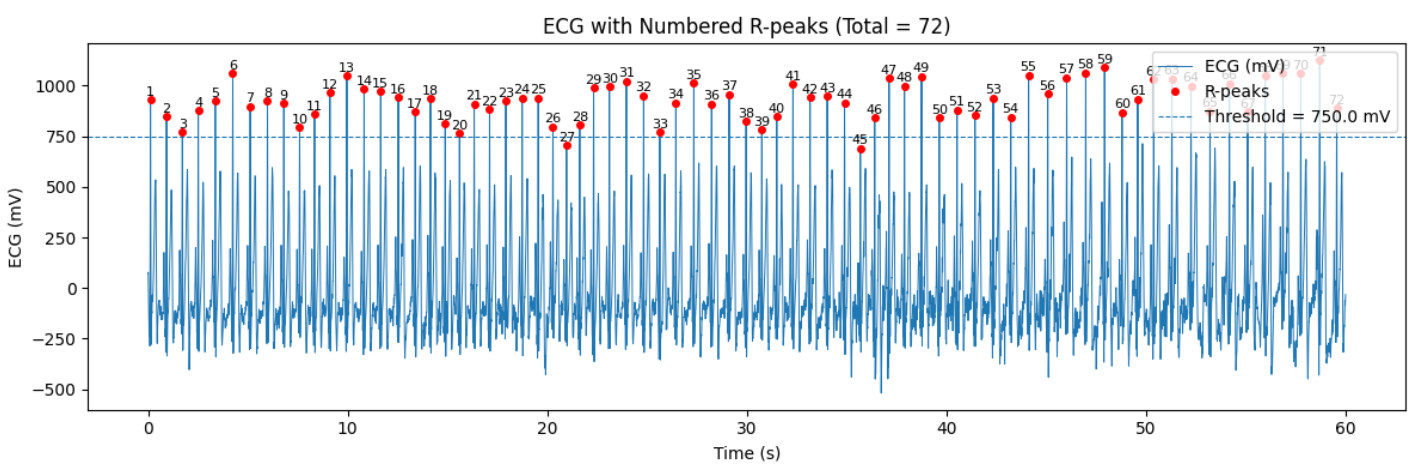

Figure 1: *ECG recording used in the analysis with detected R-peaks labeled.*

To improve visualization, the ECG was also segmented into groups of 24 beats.

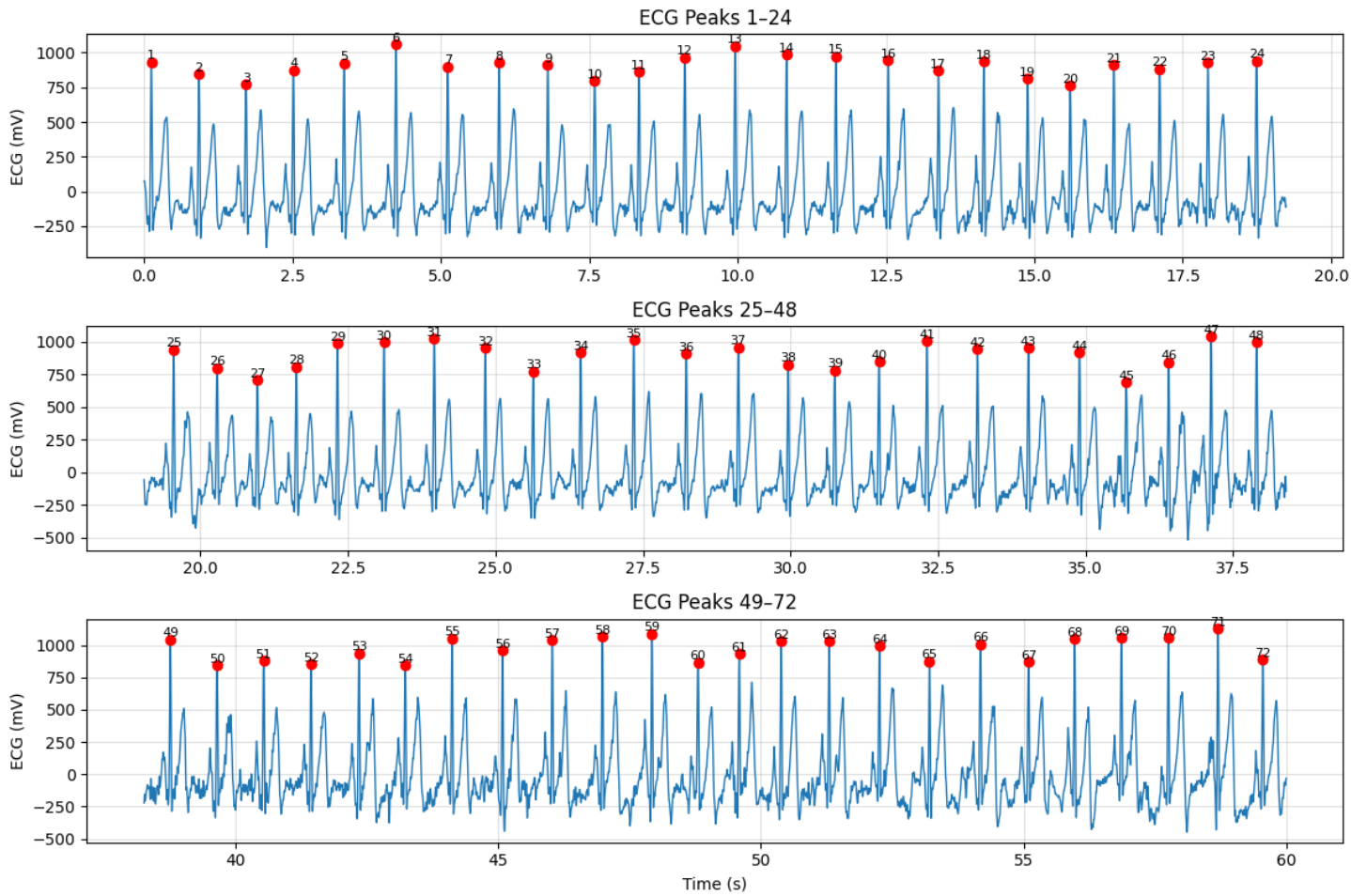

Figure 2: *Segmented ECG plots showing detected peaks.*

<u>Beat-by-Beat Heart Rate</u>

Heart rate was calculated from the RR intervals between consecutive R-peaks:

$$RR_i = t_i - t_{i-1}$$

$$HR_i = \frac{60}{RR_i}$$

where RR_i is the interval between beats in seconds and HR_i is the instantaneous heart rate in beats per minute (bpm).

The resulting values provide a beat-by-beat estimate of heart rate over the one-minute recording (Values in Appendix.A)

2. Heart Rate Variability

HR vs Time

Because beat-by-beat HR values occur at irregular time intervals, the HR signal was interpolated onto a uniform time grid for spectral analysis. The interpolated HR signal was sampled at:

$$f_s = 4 \text{ Hz}$$

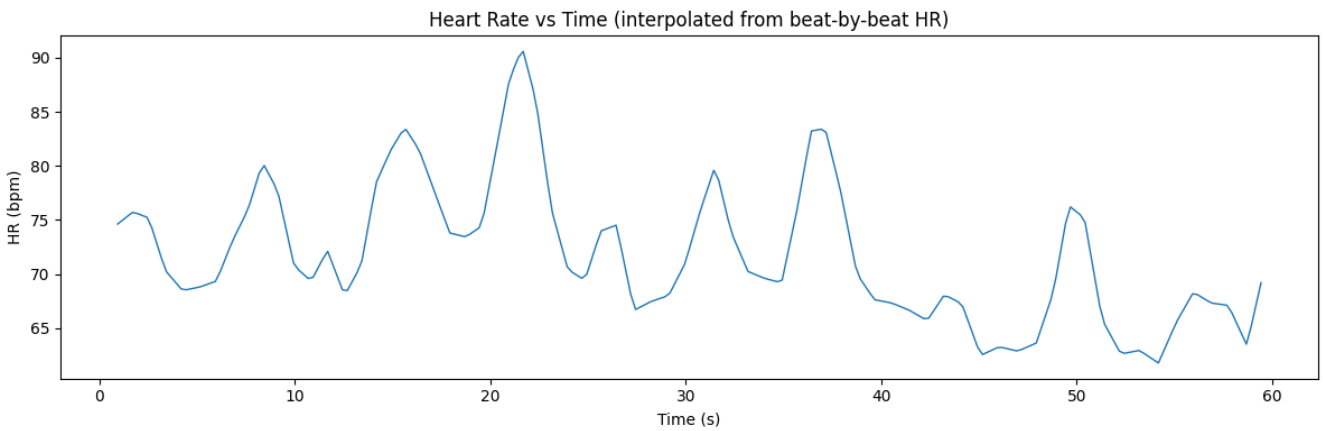

Figure 3: *Interpolated heart rate versus time.*

This plot shows the temporal variation of heart rate across the recording.

HRV Power Spectrum

The mean heart rate was removed prior to spectral analysis so that the spectrum represents only the fluctuations around the average HR.

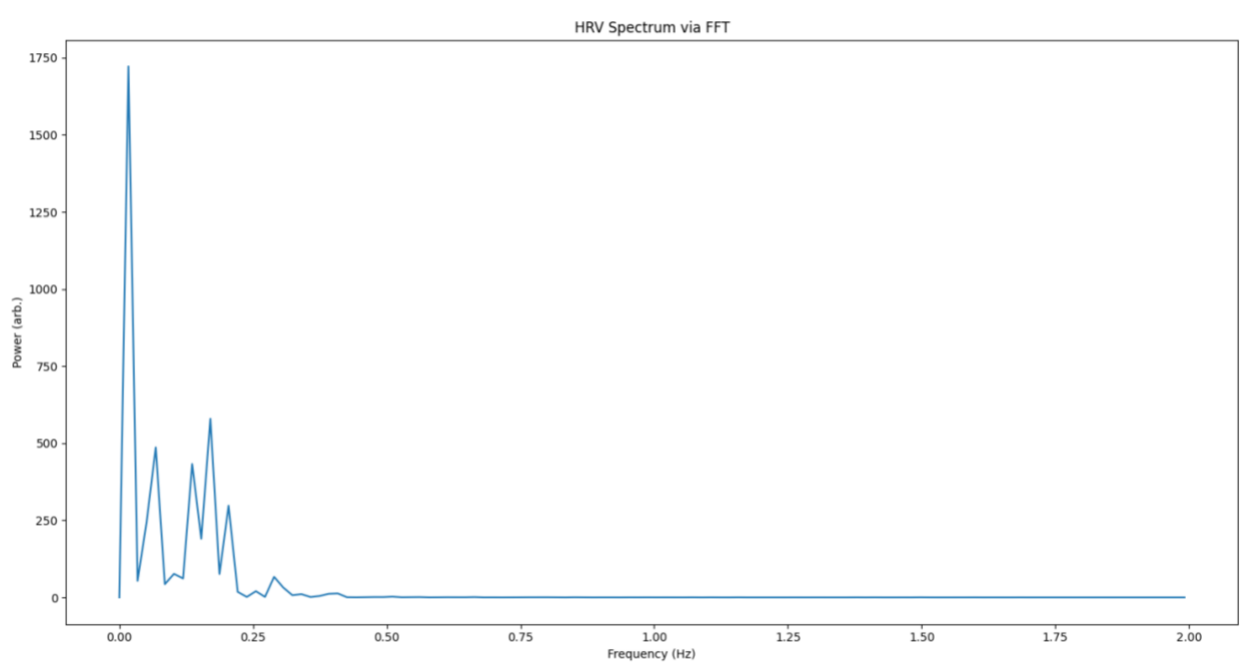

Figure 4: *Power spectrum of the heart rate signal obtained using the FFT.*

The HRV spectrum shows that most variability occurs at low frequencies ($\leq 2\ Hz$, especially the bump at ~ 0.18 Hz). These slow fluctuations are associated with the autonomic regulation of heart rate and indicative of the respiratory rate. Higher frequency components contain comparatively less power.

Conclusion

Beat-by-beat heart rate and heart rate variability were successfully computed from a one-minute ECG recording. R-peaks were detected using a threshold-based method with refractory filtering and missed-beat recovery.
Heart rate was calculated from RR intervals and interpolated to form a uniformly sampled HR signal.
Spectral analysis of this signal revealed that HR variability is primarily concentrated at frequencies around 0.15 – 0.20 Hz.

*Code used in the report can be found in Appendix.B.

Appendix

Appendix.A

Beat-by-beat HR list (bpm):

001: t = 0.925 s HR = 74.63 bpm
002: t = 1.717 s HR = 75.76 bpm
003: t = 2.515 s HR = 75.19 bpm
004: t = 3.368 s HR = 70.34 bpm
005: t = 4.244 s HR = 68.49 bpm
006: t = 5.116 s HR = 68.81 bpm
007: t = 5.981 s HR = 69.36 bpm
008: t = 6.802 s HR = 73.08 bpm
009: t = 7.592 s HR = 75.95 bpm
010: t = 8.339 s HR = 80.32 bpm
011: t = 9.111 s HR = 77.72 bpm
012: t = 9.959 s HR = 70.75 bpm
013: t = 10.824 s HR = 69.36 bpm
014: t = 11.655 s HR = 72.20 bpm
015: t = 12.537 s HR = 68.03 bpm
016: t = 13.384 s HR = 70.84 bpm
017: t = 14.149 s HR = 78.43 bpm
018: t = 14.886 s HR = 81.41 bpm
019: t = 15.604 s HR = 83.57 bpm
020: t = 16.340 s HR = 81.52 bpm
021: t = 17.112 s HR = 77.72 bpm
022: t = 17.925 s HR = 73.80 bpm
023: t = 18.742 s HR = 73.44 bpm
024: t = 19.548 s HR = 74.44 bpm
025: t = 20.285 s HR = 81.41 bpm
026: t = 20.967 s HR = 87.98 bpm
027: t = 21.627 s HR = 90.91 bpm
028: t = 22.324 s HR = 86.08 bpm
029: t = 23.113 s HR = 76.05 bpm
030: t = 23.965 s HR = 70.42 bpm
031: t = 24.829 s HR = 69.44 bpm
032: t = 25.640 s HR = 73.98 bpm
033: t = 26.445 s HR = 74.53 bpm
034: t = 27.345 s HR = 66.67 bpm

035: t = 28.234 s HR = 67.49 bpm
036: t = 29.116 s HR = 68.03 bpm
037: t = 29.961 s HR = 71.01 bpm
038: t = 30.751 s HR = 75.95 bpm
039: t = 31.501 s HR = 80.00 bpm
040: t = 32.313 s HR = 73.89 bpm
041: t = 33.167 s HR = 70.26 bpm
042: t = 34.029 s HR = 69.61 bpm
043: t = 34.896 s HR = 69.20 bpm
044: t = 35.686 s HR = 75.95 bpm
045: t = 36.407 s HR = 83.22 bpm
046: t = 37.126 s HR = 83.45 bpm
047: t = 37.897 s HR = 77.82 bpm
048: t = 38.754 s HR = 70.01 bpm
049: t = 39.641 s HR = 67.64 bpm
050: t = 40.532 s HR = 67.34 bpm
051: t = 41.432 s HR = 66.67 bpm
052: t = 42.345 s HR = 65.72 bpm
053: t = 43.226 s HR = 68.10 bpm
054: t = 44.118 s HR = 67.26 bpm
055: t = 45.078 s HR = 62.50 bpm
056: t = 46.026 s HR = 63.29 bpm
057: t = 46.980 s HR = 62.89 bpm
058: t = 47.923 s HR = 63.63 bpm
059: t = 48.801 s HR = 68.34 bpm
060: t = 49.587 s HR = 76.34 bpm
061: t = 50.385 s HR = 75.19 bpm
062: t = 51.297 s HR = 65.79 bpm
063: t = 52.255 s HR = 62.63 bpm
064: t = 53.208 s HR = 62.96 bpm
065: t = 54.179 s HR = 61.79 bpm
066: t = 55.094 s HR = 65.57 bpm
067: t = 55.972 s HR = 68.34 bpm
068: t = 56.863 s HR = 67.34 bpm
069: t = 57.757 s HR = 67.11 bpm
070: t = 58.703 s HR = 63.42 bpm
071: t = 59.557 s HR = 70.26 bpm

Appendix.B

```
import numpy as np
import matplotlib.pyplot as plt

# ==========================
# USER SETTINGS
# ==========================
fs_ecg = 1000.0          # ECG sampling rate (Hz)
thr_mV = 750.0           # initial threshold for R-peak detection (mV)
refractory_s = 0.25      # minimum time between detected peaks (s)

# RR-gap recovery (missed-beat insertion)
gap_factor = 1.4         # if RR > gap_factor * median_RR, search for a missed peak
search_margin_s = 0.12   # ignore +/- margin around interval edges when searching (s)
max_insert_passes = 5    # safety cap

# HR(t) resampling for HRV
fs_hr = 4.0              # Hz uniform grid for HR-vs-time

# Input file: ECG in mV
path = "/Users/erenerenel/Desktop/EE495/EE495 Code/ecg_final_filtered.csv"


# ==========================
# LOAD CSV
# ==========================

data = np.genfromtxt(path, delimiter=",", comments="#", invalid_raise=False)
data = np.asarray(data)

if data.ndim == 0:
    raise ValueError("CSV load failed (got scalar). Check file path / contents.")

if data.ndim == 1:
    y_mV = data.astype(float)
    t = np.arange(y_mV.size) / fs_ecg
else:
```

```
    # drop rows that are all-NaN
    data = data[~np.all(np.isnan(data), axis=1)]

    # one long row -> flatten
    if data.shape[0] == 1 and data.shape[1] > 1:
        y_mV = data.ravel().astype(float)
        t = np.arange(y_mV.size) / fs_ecg
    else:
        col0 = data[:, 0]
        ncheck = min(200, len(col0))
        is_time_like = (
            ncheck >= 10
            and np.all(np.diff(col0[:ncheck]) > 0)
            and (np.nanmedian(np.diff(col0[:ncheck])) < 0.01)
        )
        if data.shape[1] >= 2 and is_time_like:
            t = col0.astype(float)
            y_mV = data[:, 1].astype(float)
        else:
            y_mV = data[:, 0].astype(float)
            t = np.arange(y_mV.size) / fs_ecg

mask = ~np.isnan(y_mV) & ~np.isnan(t)
y_mV = y_mV[mask]
t = t[mask]

if y_mV.size < 1000:
    raise ValueError(f"Too few samples loaded: {y_mV.size}")

duration_s = t[-1] - t[0]
print(f"Loaded samples: {y_mV.size}, duration ≈ {duration_s:.2f} s")


# ==========================
# INITIAL R-PEAK DETECTION:
# threshold -> contiguous segments -> local max per segment -> refractory
# ==========================
```

```
above = y_mV > thr_mV
edges = np.diff(above.astype(int))

starts = np.where(edges == 1)[0] + 1
ends = np.where(edges == -1)[0] + 1

if above[0]:
    starts = np.r_[0, starts]
if above[-1]:
    ends = np.r_[ends, y_mV.size]

if starts.size == 0 or ends.size == 0:
    raise ValueError("No above-threshold segments found. Lower thr_mV or check units.")

# candidate peak: local max in each above-threshold segment
cand_idx = []
for s, e in zip(starts, ends):
    if e <= s:
        continue
    idx = s + int(np.argmax(y_mV[s:e]))
    cand_idx.append(idx)
cand_idx = np.array(cand_idx, dtype=int)

# refractory filter
keep = []
last_t = -np.inf
for idx in cand_idx:
    if (t[idx] - last_t) >= refractory_s:
        keep.append(idx)
        last_t = t[idx]
r_idx = np.array(keep, dtype=int)

# sort / unique just in case
r_idx = np.unique(r_idx)
r_idx = r_idx[np.argsort(t[r_idx])]

print(f"Initial detected beats: {len(r_idx)}")
```

```
# ========================
# MISSED-BEAT RECOVERY:
# If RR gap is too long (> gap_factor * median RR),
# search between peaks for the largest local maximum and insert it.
# Repeat a few passes in case multiple misses exist.
# ========================

margin = int(search_margin_s * fs_ecg)

for _pass in range(max_insert_passes):
    r_t = t[r_idx]
    if r_t.size < 3:
        break

    rr = np.diff(r_t)
    med_rr = np.median(rr)

    new_idxs = []
    for i, rr_i in enumerate(rr):
        if rr_i > gap_factor * med_rr:
            s = r_idx[i] + margin
            e = r_idx[i + 1] - margin
            if e <= s:
                continue

            # Find the strongest peak in the gap
            idx = s + int(np.argmax(y_mV[s:e]))

            # Enforce refractory vs neighboring peaks
            if (t[idx] - t[r_idx[i]]) >= refractory_s and (t[r_idx[i + 1]] - t[idx]) >= refractory_s:
                new_idxs.append(idx)

    if not new_idxs:
        break
```

```
    r_idx = np.unique(np.concatenate([r_idx, np.array(new_idxs, dtype=int)]))
    r_idx = r_idx[np.argsort(t[r_idx])]

print(f"Beats after RR-gap recovery: {len(r_idx)}")


# ==========================
# HR (beat-by-beat)
# ==========================

r_t = t[r_idx]
r_y = y_mV[r_idx]

if r_t.size < 2:
    raise ValueError("Need at least 2 peaks to compute HR. Lower thr_mV or check signal.")

rr_s = np.diff(r_t)
hr_bpm = 60.0 / rr_s
hr_t = r_t[1:]

print("\nBeat-by-beat HR list (bpm):")
for i, (ti, hri) in enumerate(zip(hr_t, hr_bpm), start=1):
    print(f"{i:03d}: t = {ti:7.3f} s   HR = {hri:6.2f} bpm")


# ==========================
# HR vs time (interpolate to uniform grid)
# ==========================

t0, t1 = hr_t[0], hr_t[-1]
t_hr_grid = np.arange(t0, t1, 1.0 / fs_hr)
hr_grid = np.interp(t_hr_grid, hr_t, hr_bpm)
hr_grid_zero = hr_grid - np.mean(hr_grid)


# ==========================
# PLOTS
# 1) ECG with numbered R-peaks
```

```
# 2) Segmented ECG peak plots
# 3) HR vs time
# 4) FFT^2
# ========================


# ========================
# FULL ECG PLOT WITH PEAKS
# ========================

plt.figure(figsize=(12, 4))
plt.plot(t, y_mV, linewidth=0.7, label="ECG (mV)")
plt.plot(r_t, r_y, "ro", markersize=4, label="R-peaks")
plt.axhline(thr_mV, linestyle="--", linewidth=0.8, label=f"Threshold = {thr_mV} mV")
for i, (ti, yi) in enumerate(zip(r_t, r_y), start=1):
    plt.text(ti, yi + 20, str(i), fontsize=8, ha="center")
plt.title(f"ECG with Numbered R-peaks (Total = {len(r_t)})")
plt.xlabel("Time (s)")
plt.ylabel("ECG (mV)")
plt.legend(loc="upper right")
plt.tight_layout()
plt.show()


# ========================
# SEGMENTED ECG PEAK PLOTS
# ========================
peaks_per_plot = 24
num_peaks = len(r_t)
num_plots = int(np.ceil(num_peaks / peaks_per_plot))

fig, axes = plt.subplots(num_plots, 1, figsize=(12, 8), sharex=False)

if num_plots == 1:
    axes = [axes]

for p in range(num_plots):
```

```
    start_peak = p * peaks_per_plot
    end_peak = min((p + 1) * peaks_per_plot, num_peaks)

    # time window from first peak to last peak in this segment
    t_start = r_t[start_peak] - 0.5
    t_end = r_t[end_peak - 1] + 0.5

    mask = (t >= t_start) & (t <= t_end)

    axes[p].plot(t[mask], y_mV[mask], linewidth=1.0)

    # plot peaks in this segment
    seg_peaks = r_idx[start_peak:end_peak]
    axes[p].plot(t[seg_peaks], y_mV[seg_peaks], "ro")

    # number peaks
    for i, idx in enumerate(seg_peaks, start=start_peak + 1):
        axes[p].text(t[idx], y_mV[idx] + 20, str(i), fontsize=8, ha='center')

    axes[p].set_title(f"ECG Peaks {start_peak+1}–{end_peak}")
    axes[p].set_ylabel("ECG (mV)")
    axes[p].grid(True, alpha=0.4)

axes[-1].set_xlabel("Time (s)")

plt.tight_layout()
plt.show()

# ========================
# HR vs time (Interpolated)
# ========================

plt.figure(figsize=(12, 4))
plt.plot(t_hr_grid, hr_grid, linewidth=1.0)
plt.title("Heart Rate vs Time (interpolated from beat-by-beat HR)")
plt.xlabel("Time (s)")
```

```
plt.ylabel("HR (bpm)")
plt.tight_layout()
plt.show()



# ========================
# FFT OF HR SIGNAL
# ========================



N = len(hr_grid_zero)
freq_fft = np.fft.rfftfreq(N, d=1/fs_hr)

fft_vals = np.fft.rfft(hr_grid_zero)
fft_mag = np.abs(fft_vals)**2 / N


print(np.mean(hr_grid_zero))
print(fft_vals[0])
dc_power = fft_mag[0]
print(f"Power at 0 Hz         = {dc_power:.6e}")


plt.figure(figsize=(12,4))
plt.plot(freq_fft, fft_mag)

plt.title("HRV Spectrum via FFT")
plt.xlabel("Frequency (Hz)")
plt.ylabel("Power (arb.)")

plt.tight_layout()
plt.show()
```